11- Longest Subarray with Equal 0s and 1s

Given a binary array (containing only 0s and 1s), find the length of the longest contiguous subarray with equal number of 0s and 1s.

Examples:
Input: nums = [0,1]
Output: 2

Input: nums = [0,1,0]
Output: 2

Input: nums = [0,0,1,0,1,1,0]
Output: 6

Write an efficient algorithm to solve this problem.

- FIRST: non-recursive algorithm

- **Algorithm 1 — HashMap + Prefix Sum**
-
- Non-Recursive · O(n) time · O(n) space
-
- **get_user_input()**

```
FUNCTION get_user_input()
▸ Display header, algorithm name, usage instructions
  LOOP indefinitely:
    READ user_input FROM standard input
    user_input ← TRIM(user_input)
    IF user_input is empty:
       DISPLAY "[Error] Input is empty. Please enter some numbers."
       CONTINUE loop
    TRY:
       nums ← CONVERT each token in user_input to integer
       IF any value in nums NOT IN {0, 1}:
          DISPLAY "[Error] Array must contain ONLY 0s and 1s."
          DISPLAY "Hint: Remove numbers like 2, 3 ... and try again."
          CONTINUE loop
       DISPLAY "[OK] Input accepted!"
       RETURN nums
    CATCH ValueError:
       DISPLAY "[Error] Invalid format — letters or symbols detected."
```

```
        DISPLAY "Hint: Use only 0s and 1s separated by spaces."
    END LOOP
END FUNCTION


longest_balanced_subarray(arr)

FUNCTION longest_balanced_subarray(arr)
    n ← LENGTH(arr)
    max_length ← 0
    start_index ← -1
    end_index ← -1
    prefix_sum ← 0
    ▸ Prefix sum = 0 is seen "before" the array starts → index = -1
    first_occurrence ← { 0 : -1 }
    FOR i FROM 0 TO n - 1:
        IF arr[i] == 0:
            prefix_sum ← prefix_sum - 1 ▸ treat 0 as -1 so equal counts cancel
        ELSE:
            prefix_sum ← prefix_sum + 1 ▸ treat 1 as +1
        IF prefix_sum EXISTS IN first_occurrence:
            current_length ← i - first_occurrence[prefix_sum]
            ▸ subarray (first_occurrence[prefix_sum]+1 .. i) is balanced
            IF current_length > max_length:
                max_length ← current_length
                start_index ← first_occurrence[prefix_sum] + 1
                end_index ← i
        ELSE:
            ▸ Store FIRST occurrence only — never overwrite
            first_occurrence[prefix_sum] ← i
    RETURN (max_length, start_index, end_index)
END FUNCTION


MAIN — Program entry

BEGIN PROGRAM
    arr ← CALL get_user_input()
    (length, start, end) ← CALL longest_balanced_subarray(arr)
    DISPLAY "Input Array :" + arr
```

```
    DISPLAY "Longest Subarray Length:" + length

    IF start != -1:
        DISPLAY "Subarray Indices : [" + start + " .. " + end + "]"
        DISPLAY "Subarray :" + arr[start .. end]
    ELSE:
        DISPLAY "No valid subarray found."
END PROGRAM
```

ANALYSIS : non-recursive algorithm

- Algorithm Complexity Analysis
- 1. Time Complexity Analysis
- Let the input size be n.
- The function contains a single loop: for i = 0 to n-1. Therefore, the number of iterations is n.
- Work performed inside each iteration:
- ● Checking arr[i] == 0 → O(1)
- ● Updating Sum → O(1)
- ● Checking if Sum exists in Hash Map → O(1)
- ● Calculating Subarray length → O(1)
- ● Updating Maximum length → O(1)
- ● Inserting into Hash Map (if needed) → O(1)
- Total work per iteration:
- O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)
- Total Time & Recurrence Relation
- Approach Calculation
- Iterative Calculation T(n) = n × O(1) = O(n)
- Recurrence Relation T(n) = T(n-1) + c (where c is constant)
- Expanding the Recurrence:

- • T(n) = T(n-1) + c
- • T(n) = T(n-2) + 2c
- • T(n) = T(n-3) + 3c
- • ...
- • T(n) = T(0) + nc
- Since T(0) = c, then T(n) = c + nc = c(n + 1). Final Time Complexity: T(n) = O(n).
-
- 2. Space Complexity Analysis
-
- To execute the algorithm, we use:
- ● Variables: Uses constant space → O(1)
- ● Hash Map (SumToIndex):
- ○ In the worst case, each prefix sum is unique.
- ○ Therefore, we store n values → O(n)
- Total Space:
- S(n) = O(n) + O(1)
- S(n) = O(n)
-
- Final Clean Answer
-
- ■ Time Complexity: T(n) = n × O(1) = O(n)
- ■ Recurrence Relation: T(n) = T(n-1) + c ⇒ T(n) = O(n)
- ■ Space Complexity: S(n) = O(n)

- SECOND: recursive algorithm

Algorithm 2 — Divide & Conquer

Recursive · $O(n^2)$ time · $O(\log n)$ space

get_user_input()

```
FUNCTION get_user_input()
    ▸ Display header, algorithm name, usage instructions
    LOOP indefinitely:
        READ user_input FROM standard input
        user_input ← TRIM(user_input)

        IF user_input is empty:
            DISPLAY "[Error] Input is empty. Please enter some numbers."
            CONTINUE loop
        TRY:
            nums ← CONVERT each token in user_input to integer
            IF any value in nums NOT IN {0, 1}:
                DISPLAY "[Error] Array must contain ONLY 0s and 1s."
                DISPLAY "Hint: Remove numbers like 2, 3 ... and try again."
                CONTINUE loop
            DISPLAY "[OK] Input accepted!"
            RETURN nums
        CATCH ValueError:
            DISPLAY "[Error] Invalid format — letters or symbols detected."
            DISPLAY "Hint: Use only 0s and 1s separated by spaces."
    END LOOP
END FUNCTION
```

longest_subarray_recursive(nums)

```
FUNCTION longest_subarray_recursive(nums)
    n ← LENGTH(nums)
    IF n < 2:
        RETURN (0, -1, -1) ▸ array too short to ever be balanced
    RETURN CALL divide_and_conquer(nums, 0, n - 1)
END FUNCTION
```

divide_and_conquer(arr, left, right)

```
FUNCTION divide_and_conquer(arr, left, right)
    ▸ Base case: a single element can never be balanced
    IF left >= right:
        RETURN (0, -1, -1)
    mid ← left + (right - left) / 2 ▸ integer division, avoids overflow
     ▸ Step 1: Divide — recurse on left half [left .. mid]
     (left_len, left_s, left_e) ← CALL divide_and_conquer(arr, left, mid)
     ▸ Step 2: Divide — recurse on right half [mid+1 .. right]
     (right_len, right_s, right_e) ← CALL divide_and_conquer(arr, mid + 1, right)
     ▸ Step 3: Conquer — find best subarray crossing the midpoint
     (cross_len, cross_s, cross_e) ← CALL combine_crossing(arr, left, mid, right)
     ▸ Return the best among the three candidates
     IF left_len >= right_len AND left_len >= cross_len:
        RETURN (left_len, left_s, left_e)
     ELSE IF right_len >= left_len AND right_len >= cross_len:
        RETURN (right_len, right_s, right_e)
     ELSE:
        RETURN (cross_len, cross_s, cross_e)
END FUNCTION
```

combine_crossing(arr, left, mid, right)

```
FUNCTION combine_crossing(arr, left, mid, right)
    max_len ← 0
    best_start ← -1
    best_end ← -1

    ▸ Scan every possible left boundary i (i ≤ mid < j)
    FOR i FROM left TO mid:
        zeros ← 0
        ones ← 0

       ▸ Build counts from i to mid (left side) — rebuilt each outer iteration
       FOR k FROM i TO mid:
          IF arr[k] == 0: zeros ← zeros + 1
```

```
        ELSE: ones ← ones + 1

        ▸ Extend right side one element at a time
        FOR j FROM mid + 1 TO right:
            IF arr[j] == 0: zeros ← zeros + 1
            ELSE: ones ← ones + 1

            IF zeros == ones:
                current_len ← j - i + 1
                IF current_len > max_len:
                    max_len ← current_len
                    best_start ← i
                    best_end ← j
    RETURN (max_len, best_start, best_end)
END FUNCTION


MAIN — Program entry

BEGIN PROGRAM
    arr ← CALL get_user_input()
    (length, start, end) ← CALL longest_subarray_recursive(arr)

    DISPLAY "Input Array :" + arr
    DISPLAY "Longest Subarray Length:" + length

    IF start != -1:
        DISPLAY "Subarray Indices : [" + start + " .. " + end + "]"
        DISPLAY "Subarray :" + arr[start .. end]
    ELSE:
        DISPLAY "No valid subarray found."
END PROGRAM
```

- ANALYSIS : recursive algorithm

- Algorithm Complexity Analysis (Case 2)
- 1. Time Complexity Analysis
-
- Let the input size be n.
- The function structure is as follows:
- ● Outer loop: number of iterations ≈ n/2
- ● Inside loop 2:
- ○ For k from i to mid → O(n)
- ○ For j from mid + 1 to right → O(n)
- ● Total work inside: O(n) + O(n) = O(n)
- ● Total work: n/2 × O(n) = O(n^2)
- Recurrence Relation
- The total time can be expressed as:
- T(n) = 2T(n/2) + O(n^2)
- Or: T(n) = 2T(n/2) + cn^2
- Expanding the Recurrence:
- • T(n) = 2T(n/2) + n^2
-
- • Substituting T(n/2) = 2T(n/4) + (n/2)^2
-
- • T(n) = 2[2T(n/4) + n^2
- /4] + n^2
-
- • T(n) = 4T(n/4) + n^2
- /2 + n^2
-
- After k times:
- T(n) = 2^k T(n/2^k) + n^2 ∑i=0
- k-1
- (1/2)^i
-
- Since the summation ∑ (1/2)^i
- is a constant:
-

- Final Time Complexity: T(n) = O(n^2)
-
- 2. Space Complexity Analysis
- ●
- Variables: max_len, best_start, best_end → O(1)
- ● Recursion Stack:
- ○ Since we divide the array by 2 each time, the depth of the recursion tree is log n.
- Total Space:
-
- S(n) = O(1) + O(log n)
-
- Final Space Complexity: S(n) = O(log n

- Comparison

• Feature	Algorithm 1: HashMap + Prefix Sum	Algorithm 2: Divide & Conquer
Core Logic	Uses a Prefix Sum technique where 0 is treated as -1 and 1 as +1 to find indices where the cumulative sum repeats.	Uses a Recursive approach to split the array into halves and check left, right, and crossing segments.
Time Complexity	O(n) (Linear Time) — The array is traversed only once.	O(n^2) (Quadratic Time) — The combine_crossing function uses nested loops to check boundaries.
Space Complexity	O(n) — Requires a Hash Map to store the first occurrence of each prefix sum.	O(log n) — Consumes space on the recursion stack based on the depth of the tree.
Mechanism	Iterative: Maintains a running sum and checks a dictionary for previously seen values.	Recursive: Breaks the problem into smaller sub-problems until a base case (length < 2) is reached.
Data Structures	Uses a HashMap (Dictionary) to store {prefix_sum : index} pairs.	Uses the Function Call Stack for recursion and simple variables for indexing.
Efficiency	Highly Efficient: Ideal for very large input arrays due to its linear nature.	Less Efficient: Performance degrades significantly as the input size (n) increases due to nested loops.
Input Validation	Ensures input contains only 0s and 1s and is not empty.	Also ensures input contains only 0s and 1s and is not empty.